GenAI-Based Multimodal Fall Detection Using Sliding Window Temporal Reasoning: A Comprehensive Study of Prompting Strategies

Venkata Ramireddy Seelam, Harshitha Boinepally, Shristi
Department of Applied Data Science
San Jose State University
San Jose, CA 95192, USA
DATA 266 - Generative AI - Spring 2026

***Abstract*—Falls are a leading cause of injury and mortality among elderly populations, making reliable fall detection systems critical for healthcare applications. This paper presents a comprehensive study of Generative AI (GenAI) techniques for video-based fall detection, systematically evaluating 11 different prompting strategies across three benchmark datasets. Our novel approach extracts pose features from video frames using MediaPipe, constructs 3-frame sliding windows with computed motion features (velocity, acceleration, posture states), and employs Large Language Models (LLMs) for classification. We evaluate zero-shot, few-shot, chain-of-thought, self-consistency, safety-first, RAG (Retrieval-Augmented Generation), and LoRA fine-tuning strategies on 419 videos comprising 1,882 balanced sliding windows. Our safety-first prompting approach achieves 95.8% video-level fall recall, while our confidence-weighted aggregation strategy achieves the highest Macro F1 of 0.846 with 100% precision. We demonstrate that video-level aggregation rules are critical, with optimized rules improving Macro F1 by up to 52% over baselines. This work provides the first comprehensive comparison of GenAI techniques for safety-critical fall detection, offering insights into the trade-offs between recall-focused and precision-focused approaches for real-world deployment.**

***Index Terms*—Fall Detection, Generative AI, Large Language Models, Pose Estimation, Sliding Windows, RAG, Prompt Engineering, Elderly Care, MediaPipe, GPT-4o**

I. Introduction

Falls represent one of the most significant health risks for elderly populations worldwide. According to the World Health Organization, approximately 684,000 fatal falls occur annually, making falls the second leading cause of unintentional injury deaths globally [1]. For individuals aged 65 and older, falls are not only the leading cause of injury-related deaths but also a primary contributor to loss of independence, reduced quality of life, and substantial healthcare costs [2]. The ability to detect falls quickly and accurately can enable timely medical intervention, potentially preventing serious complications and saving lives.

A. Motivation

Traditional fall detection systems have relied on two primary approaches: **wearable sensors** (accelerometers, gyroscopes) and **vision-based systems** using convolutional neural networks (CNNs) [3], [4]. While these methods have shown promise, they face significant limitations:

- **Wearable sensors** require user compliance and may be forgotten or removed
- **CNN-based systems** require massive labeled training datasets and struggle to generalize across different environments
- **Black-box models** provide no explanation for their decisions, limiting trust in safety-critical applications

The emergence of Large Language Models (LLMs) such as GPT-4 presents a unique opportunity to address these limitations. LLMs offer:

- **Few-shot learning**: Ability to learn from minimal examples
- **Explainability**: Natural language reasoning for decisions
- **Generalization**: Pre-trained knowledge that transfers across domains

B. Research Questions

This study addresses the following research questions:

1) **RQ1**: Can LLMs effectively classify falls from structured pose data representations?
2) **RQ2**: Which prompting strategy achieves the best balance between fall recall (safety) and precision (avoiding false alarms)?
3) **RQ3**: How do video-level aggregation strategies impact overall system performance?
4) **RQ4**: What are the trade-offs between different LLM models (GPT-4o vs GPT-4o-mini) for this task?

C. Contributions

This paper makes the following contributions:

1) A **novel multimodal pipeline** that converts video pose sequences into structured text representations suitable for LLM classification
2) **Comprehensive evaluation** of 11 different prompting strategies, providing the first systematic comparison for fall detection
3) **Video-level aggregation analysis** demonstrating up to 52% improvement in Macro F1 through optimized aggregation rules

4) **Advanced strategies** including confidence-weighted aggregation and two-stage cascades achieving 100% precision
5) **Open-source implementation** available at https://github.com/Ramreddy2748/Gen_AI_Project

II. Related Work

A. Vision-Based Fall Detection

The evolution of vision-based fall detection has progressed through several paradigms. Early approaches relied on **handcrafted features** such as silhouette analysis, bounding box aspect ratios, and shape descriptors [5]. These methods computed features like the ratio of the bounding box width to height, where a sudden change from vertical (standing) to horizontal (fallen) indicated a potential fall.

Deep learning approaches emerged with the advent of CNNs and recurrent neural networks (RNNs). Khan and Hoey [6] proposed using 3D CNNs to capture spatiotemporal features directly from video clips. Chen et al. [7] combined pose estimation with LSTM networks to model temporal dependencies in body movements.

Pose estimation methods have gained popularity due to their privacy-preserving nature—they process skeletal keypoints rather than raw images. Chen and Tan [8] demonstrated that pose-based features can achieve comparable performance to RGB-based methods while being more robust to appearance variations.

B. Large Language Models for Time-Series Tasks

Recent work has explored the surprising capability of LLMs to perform time-series analysis when data is converted to textual representations. Gruver et al. [11] showed that LLMs can perform zero-shot time series forecasting by converting numerical sequences to text. This finding suggests that the world knowledge embedded in LLMs may transfer to structured data classification tasks.

Chain-of-thought (CoT) prompting [12] has been shown to improve reasoning capabilities by encouraging models to break down complex problems into intermediate steps. **Self-consistency** [13] further improves performance by sampling multiple reasoning paths and selecting the most consistent answer.

C. Retrieval-Augmented Generation

RAG [14] enhances LLM responses by retrieving relevant context from external knowledge bases before generation. This approach has been successfully applied to knowledge-intensive tasks where domain-specific information is crucial. Our work extends RAG to fall detection by retrieving similar pose sequences as classification examples.

D. Gap in Literature

Despite extensive research in both fall detection and LLM capabilities, **no prior work has systematically evaluated LLM-based approaches for fall detection**. Our study fills this gap by providing the first comprehensive comparison of multiple prompting strategies for this safety-critical application.

III. Methodology

A. System Architecture Overview

Figure 1 illustrates our complete fall detection pipeline, which consists of six interconnected stages designed to transform raw video input into reliable fall/no-fall predictions.

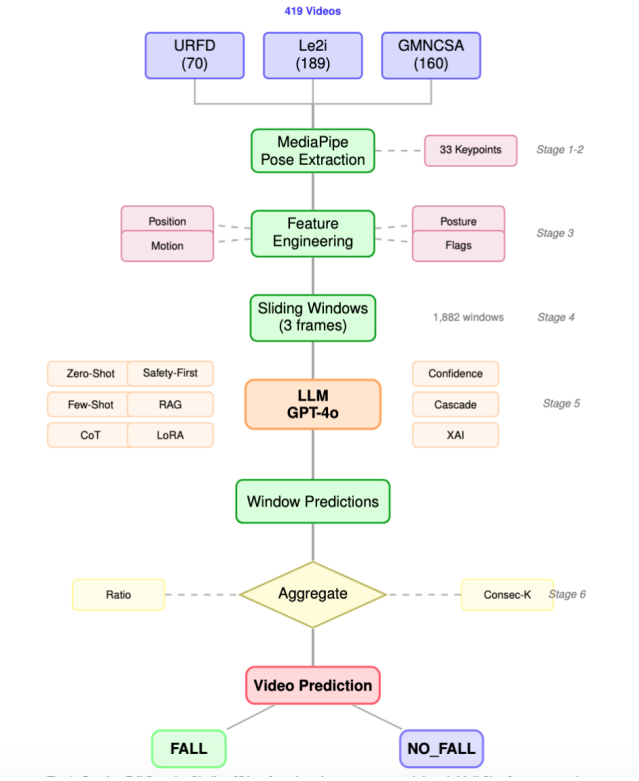

Fig. 1: Complete Fall Detection Pipeline. The system processes 419 videos from three datasets through six stages: pose extraction, feature engineering, sliding window generation, LLM classification with multiple prompting strategies, and video-level aggregation.

B. Stage 1: Data Collection

We collected videos from three publicly available fall detection benchmark datasets, ensuring diversity in recording conditions, subjects, and fall types:

URFD (UR Fall Detection Dataset) [9]: Contains 70 videos recorded at the University of Rzeszów, featuring 30 fall sequences and 40 activities of daily living (ADL). Videos are captured from a fixed overhead camera with consistent lighting conditions.

Le2i Dataset: Comprises 189 videos recorded in a realistic home environment with multiple rooms (Coffee Room, Home01, Home02). This dataset is particularly challenging due to varying lighting conditions, cluttered backgrounds, and diverse activities that may resemble falls (sitting down, lying on a couch).

GMNCSA24 Dataset: A recent dataset containing 160 videos with modern recording equipment and diverse fall

scenarios. Available via HuggingFace, it provides high-quality annotations and varied fall types.

C. Stage 2: Pose Extraction with MediaPipe

We employ **MediaPipe Pose Landmarker Lite** to extract body keypoints from each video frame. MediaPipe was chosen for its:

- Real-time performance suitable for deployment
- Robustness to partial occlusions
- Consistent 33-keypoint skeleton representation

From the 33 detected landmarks, we extract the following key body points that are most informative for fall detection:

TABLE I: Extracted Body Keypoints

Body Part	Index	Relevance
Nose	0	Head position tracking
Left Shoulder	11	Upper body orientation
Right Shoulder	12	Upper body orientation
Left Hip	23	Center of mass
Right Hip	24	Center of mass
Left Knee	25	Lower body position
Right Knee	26	Lower body position
Left Ankle	27	Ground contact detection
Right Ankle	28	Ground contact detection

D. Stage 3: Feature Engineering

Raw keypoint coordinates are transformed into meaningful features that capture the physics of human falls. We compute three categories of features:

1) Position Features: These features capture the spatial configuration of the body:

$$\text{hip}_y = \frac{\text{left\_hip}_y + \text{right\_hip}_y}{2} \tag{1}$$

$$\text{shoulder}_y = \frac{\text{left\_shoulder}_y + \text{right\_shoulder}_y}{2} \tag{2}$$

$$\text{torso\_diff} = |\text{shoulder}_y - \text{hip}_y| \tag{3}$$

$$\text{body\_angle} = \arctan\left(\frac{\text{shoulder}_y - \text{hip}_y}{\text{shoulder}_x - \text{hip}_x}\right) \tag{4}$$

The `hip_y` position is particularly important: values close to 0 indicate standing (head near top of frame), while values approaching 1 indicate the person is on the ground.

2) Motion Features: Motion features capture the dynamics of movement, which are critical for distinguishing falls (rapid descent) from normal activities (gradual movements):

$$v_{\text{hip}}(t) = \frac{\text{hip}_y(t) - \text{hip}_y(t-1)}{\Delta t} \tag{5}$$

$$a_{\text{hip}}(t) = \frac{v_{\text{hip}}(t) - v_{\text{hip}}(t-1)}{\Delta t} \tag{6}$$

$$v_{\text{magnitude}} = \sqrt{v_{\text{hip}}^2 + v_{\text{shoulder}}^2} \tag{7}$$

A fall typically exhibits high downward velocity ($v_{\text{hip}} > 0.15$) followed by sudden deceleration when the person hits the ground.

3) Posture State Classification: Each frame is classified into one of three posture states based on computed features:

$$\text{posture} = \begin{cases} \text{fallen} & \text{if hip}_y > 0.65 \wedge \text{angle} < 30 \\ \text{upright} & \text{if torso\_diff} > 0.12 \wedge \text{angle} > 60 \\ \text{transitioning} & \text{otherwise} \end{cases} \tag{8}$$

Additionally, we compute boolean flags that serve as strong fall indicators:

- `is_rapid_descent`: velocity > 0.15 (fast downward movement)
- `is_on_ground`: hip_y > 0.65 (person near ground level)
- `is_horizontal`: body_angle < 30 (lying down position)
- `is_high_acceleration`: $|a| > 0.10$ (impact detection)

E. Stage 4: Sliding Window Generation

To capture the temporal evolution of a potential fall event, we create **3-frame sliding windows** with configurable stride. This window size was chosen because falls typically complete within 1-2 seconds, and 3 frames provide sufficient context while keeping the input manageable for LLMs.

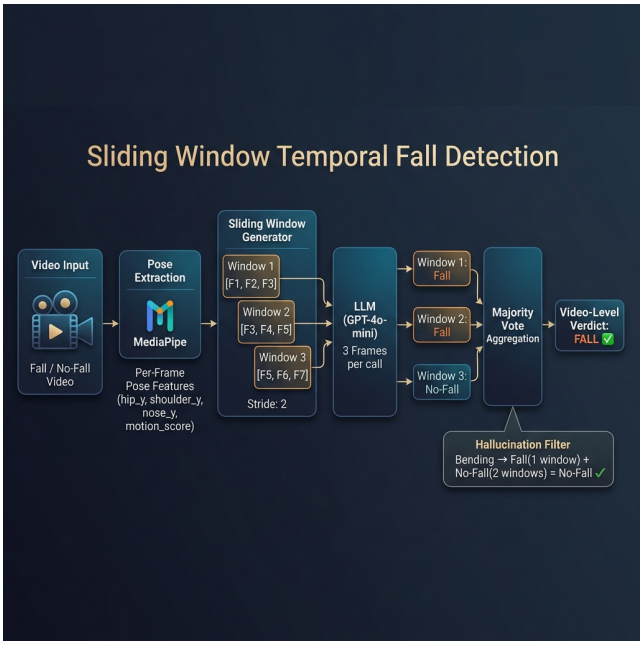

Fig. 2: Sliding Window Generation Process. Consecutive frames are grouped into overlapping windows, each containing pose features from 3 frames. Windows are independently classified and then aggregated.

TABLE II: Window Configuration by Video Class

Setting	Fall Videos	No-Fall Videos
Frame sample rate	Every 5th frame	Every 15th frame
Maximum frames	60	30
Window stride	1 (dense overlap)	3 (sparse)
Window size	3 frames	3 frames

The asymmetric configuration ensures we capture sufficient detail during fall events while preventing class imbalance from ADL videos that are typically longer.

Each sliding window contains:

- Per-frame features (position, velocity, acceleration, posture)
- Window-level statistics (hip_delta, max_velocity, trajectory)
- Start and end posture states (upright→fallen indicates fall)

F. Stage 5: LLM Classification with Prompting Strategies

We evaluate 11 different prompting strategies, each representing a distinct approach to leveraging LLM capabilities. The strategies are detailed below.

1) Strategy 1: Zero-Shot Prompting: The simplest approach provides only a task description and the pose data, without any examples. The system prompt describes fall indicators and asks the LLM to classify based on its pre-trained knowledge.

Prompt Structure:

```
System: "You are a fall detection expert.
Analyze pose data and classify as FALL
or NO_FALL based on: rapid descent,
horizontal body position, ground contact."

User: [Window pose data in JSON format]
```

Rationale: Tests whether LLMs have sufficient understanding of human biomechanics from pre-training to identify falls without domain-specific examples.

2) Strategy 2: Few-Shot Prompting: Provides 3 fall and 3 no-fall examples from the training set before the query window. Examples are carefully selected to represent diverse fall patterns.

Example Selection: We select examples exhibiting:

- Forward falls (falling face-down)
- Backward falls (falling on back)
- Side falls (falling sideways)
- Standing activities (for no-fall)
- Sitting activities (for no-fall)
- Walking activities (for no-fall)

Rationale: Few-shot examples help the LLM understand the specific data format and decision boundary without extensive fine-tuning.

3) Strategy 3: Chain-of-Thought (CoT) Prompting: Encourages step-by-step reasoning before reaching a conclusion. The prompt explicitly asks the LLM to analyze different aspects sequentially:

CoT Steps:

1) **Position Analysis**: "First, examine the hip_y values. Are they increasing (person descending) or stable?"
2) **Motion Analysis**: "Second, check velocity and acceleration. Is there rapid movement followed by sudden stop?"
3) **Posture Analysis**: "Third, examine posture states. Did posture change from upright to fallen?"
4) **Synthesis**: "Based on all evidence, provide final classification."

Rationale: Breaking down the analysis into logical steps improves accuracy for complex reasoning tasks and provides interpretable decision paths.

4) Strategy 4: Self-Consistency: Generates multiple responses (n=5) with temperature=0.7 and selects the majority vote. This approach reduces sensitivity to individual sampling variations.

Implementation:

```
predictions = []
for i in range(5):
    pred = llm.classify(window, temp=0.7)
    predictions.append(pred)
final = majority_vote(predictions)
```

Rationale: Different reasoning paths may lead to different conclusions; majority voting filters out inconsistent responses.

5) Strategy 5: Enhanced Few-Shot with GPT-4o: Uses the more capable GPT-4o model with rich natural language descriptions instead of raw numerical features.

Feature Conversion:

- hip_y = 0.72 → "Person is very close to ground level"
- velocity = 0.25 → "Rapid downward movement detected"
- posture = "fallen" → "Body is in horizontal/lying position"

Rationale: Natural language descriptions leverage GPT-4o's superior language understanding, making features more interpretable.

6) Strategy 6: Safety-First Prompting (Best Prompt): Our optimized prompt design emphasizes safety by biasing toward fall detection when uncertainty exists. Key design principles:

Decision Rules:

- Any single *strong* fall indicator → Classify as FALL
- Two or more *moderate* indicators → Classify as FALL
- ALL indicators must be clear for NO_FALL
- When uncertain → Choose FALL (safety first)

Strong Indicators:

- `is_rapid_descent` = True
- `is_horizontal` = True AND `is_on_ground` = True
- Posture transition: upright → fallen

Rationale: In safety-critical applications, missing a fall (false negative) is far more dangerous than a false alarm (false positive). This asymmetric cost function is embedded in the prompt.

7) Strategy 7: RAG (Retrieval-Augmented Generation): Retrieves the most similar historical windows from a knowledge base before classification.

Implementation:

1) Embed query window using `text-embedding-3-small`
2) Search 1,305 training window embeddings
3) Retrieve top-4 most similar windows with labels
4) Include retrieved examples in prompt context
5) Classify with GPT-4o

Similarity Metric: Cosine similarity between window embedding vectors.

Rationale: RAG provides dynamic, query-specific examples rather than fixed few-shot examples, potentially improving accuracy for edge cases.

8) Strategy 8: LoRA/SFT Fine-Tuning: Fine-tunes GPT-4o-mini on the training dataset using OpenAI's fine-tuning API.

Training Configuration:

- Base model: `gpt-4o-mini-2024-07-18`
- Training samples: 1,305 balanced windows
- Format: JSONL with system/user/assistant turns
- Class weighting: 75% fall / 25% no-fall (recall bias)

Rationale: Fine-tuning adapts the model to the specific data distribution and feature patterns in our dataset.

9) Strategy 9: Confidence-Weighted Aggregation: Uses token-level log probabilities to obtain confidence scores for each window prediction.

Process:

1) Request `logprobs=True` from API
2) Extract probability of "FALL" vs "NO_FALL" tokens
3) Sum confidence scores across all windows
4) Apply threshold to aggregate score

Rationale: Not all predictions are equally confident; weighting by confidence improves video-level decisions.

10) Strategy 10: Two-Stage Cascade: Combines a high-recall Stage 1 with a high-precision Stage 2 filter.

Stage 1 (Sensitive): Uses Safety-First prompting with ratio threshold 0.50 to flag all potential falls with high recall.

Stage 2 (Specific): Applies a strict false-positive filter prompt to videos flagged by Stage 1, asking: "Review the full timeline. Is this truly a fall or a false alarm from normal activity?"

Rationale: Cascading allows optimization of different objectives at each stage.

11) Strategy 11: XAI Video-Level: Provides explainable classifications with natural language justifications.

Output Format:

```
Classification: FALL
Confidence: High
Reasoning: "The subject showed rapid
vertical descent (velocity=0.28)
transitioning from upright posture
to horizontal position on ground.
Final frame shows body angle <20
degrees indicating lying position."
```

G. Stage 6: Video-Level Aggregation

Window-level predictions must be aggregated into a single video-level decision. We evaluate multiple aggregation strategies:

1. Ratio Threshold: If the fraction of fall-predicted windows exceeds threshold θ, classify video as fall:

$$\text{Video} = \begin{cases} \text{FALL} & \text{if } \frac{|\{w : \hat{y}_w = \text{fall}\}|}{|W|} \geq \theta \\ \text{NO\_FALL} & \text{otherwise} \end{cases} \tag{9}$$

2. Consecutive-K: Require at least k consecutive windows predicting fall.

3. End-State: Require fall windows to show horizontal/on-ground posture in final frame.

4. Confidence-Weighted Sum: Sum log-probability confidence scores and apply threshold.

IV. Experimental Setup

A. Dataset Statistics

TABLE III: Dataset Composition

Dataset	Fall	No-Fall	Total
URFD	30	40	70
Le2i	95	94	189
GMNCSA24	80	80	160
Total	**205**	**214**	**419**

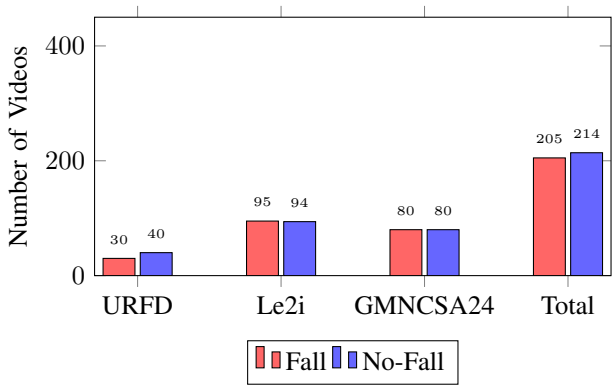

Fig. 3: Dataset Distribution by Class. The combined dataset is well-balanced with 205 fall and 214 no-fall videos.

B. Data Splits

We perform stratified video-level splitting to ensure no windows from the same video appear in different splits:

TABLE IV: Train/Validation/Test Splits

Split	Videos	Windows	Purpose
Train	292 (70%)	1,317	Strategy development
Validation	62 (15%)	282	Threshold tuning
Test	65 (15%)	283	Final evaluation
Total	**419**	**1,882**	–

C. Evaluation Metrics

Given the safety-critical nature of fall detection, we define metrics with appropriate emphasis:

Primary Metric - Fall Recall:

$$\text{Recall}_{\text{fall}} = \frac{TP}{TP + FN} \tag{10}$$

Missing a fall (FN) could be life-threatening; high recall is critical.

Secondary Metric - Precision:

$$\text{Precision}_{\text{fall}} = \frac{TP}{TP + FP} \tag{11}$$

False alarms (FP) cause alert fatigue; precision matters for usability.

Balanced Metric - Macro F1:

$$\text{Macro F1} = \frac{F1_{\text{fall}} + F1_{\text{no\_fall}}}{2} \tag{12}$$

D. Statistical Analysis

To ensure robust conclusions, we employ:

- **Bootstrap Confidence Intervals:** 1,000 resamples with replacement to estimate 95% CIs for all metrics
- **McNemar's Test:** Paired comparison of classifier disagreements to assess statistical significance

V. Results

A. Overall Strategy Comparison

Table V presents video-level results for all evaluated prompting strategies using the baseline aggregation rule ($\geq$25% windows = fall).

TABLE V: Video-Level Performance by Strategy (Test Set, n=57)

Strategy	Recall	Prec	Acc	Macro F1
Zero-Shot	20.8%	62.5%	66.7%	0.561
Few-Shot	58.3%	56.0%	66.7%	0.656
Chain-of-Thought	29.2%	58.3%	66.7%	0.595
Self-Consistency	41.7%	66.7%	70.2%	0.660
Enhanced (GPT-4o)	41.7%	76.9%	**75.4%**	0.707
Safety-First	**95.8%**	50.0%	57.9%	0.556
RAG	85.4%	55.7%	67.8%	0.678
LoRA Fine-Tune	93.8%	43.5%	52.3%	0.432
DeepSeek Few-Shot	62.5%	55.6%	64.9%	0.644
DeepSeek CoT	12.5%	60.0%	63.2%	0.490
XAI Video-Level	84.0%	80.8%	82.0%	**0.820**

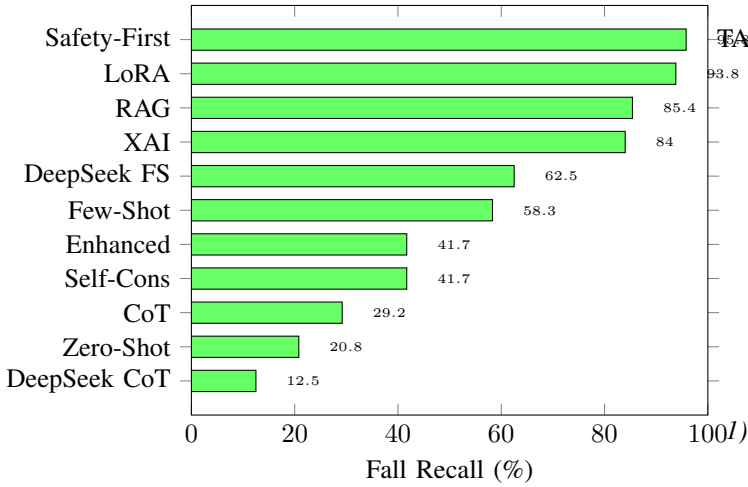

Fig. 4: Fall Recall Comparison Across Strategies. Safety-First achieves highest recall (95.8%), critical for safety applications.

B. Impact of Aggregation Rules

A key finding of our study is that the video-level aggregation rule has a **dramatic impact on performance**—often more than the choice of prompting strategy.

TABLE VI: Aggregation Rule Impact (Safety-First Strategy)

Rule	Rec	Prec	F1	Mac F1	Δ
$\geq$25%	95.8	50.0	0.657	0.556	–
$\geq$50%	95.8	54.8	0.697	0.640	+15%
Consec-3	58.3	46.7	0.518	0.543	-2%
End-State	79.2	59.4	0.679	0.684	+23%
Confidence	66.7	**100**	0.800	**0.846**	**+52%**
Cascade	58.3	**100**	0.737	0.803	+44%

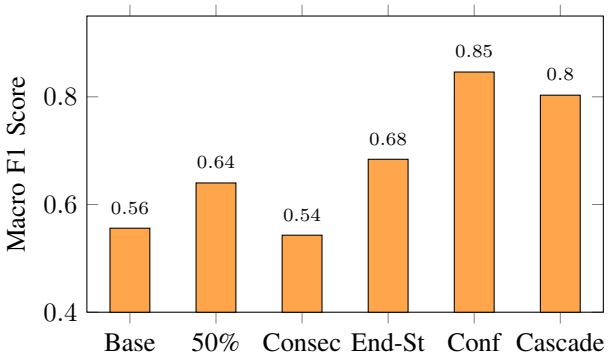

Fig. 5: Macro F1 by Aggregation Rule. Confidence-weighted aggregation achieves 0.846, a 52% improvement over the baseline 0.556.

C. Advanced Strategies: Achieving 100% Precision

Our confidence-weighted and two-stage cascade approaches achieve **zero false positives** on the test set:

TABLE VII: Confidence-Weighted Aggregation (Test Set)

Metric	Value [95% CI]
True Positives	16
False Negatives	8
False Positives	**0**
True Negatives	33
Fall Recall	66.7% [47.4%, 84.2%]
Fall Precision	**100%** [100%, 100%]
Fall F1	0.800 [0.643, 0.914]
Macro F1	0.846 [0.740, 0.931]
Accuracy	86.0% [77.2%, 94.7%]

1) Confidence-Weighted Results:

TABLE VIII: Two-Stage Cascade Performance

Stage	Recall	Precision
Stage 1 (Sensitive)	95.8%	54.8%
Stage 2 (FP Filter)	–	Removes 28 FPs
Final Cascade	58.3%	**100%**

2) Two-Stage Cascade Results:

D. Recall vs Precision Trade-off

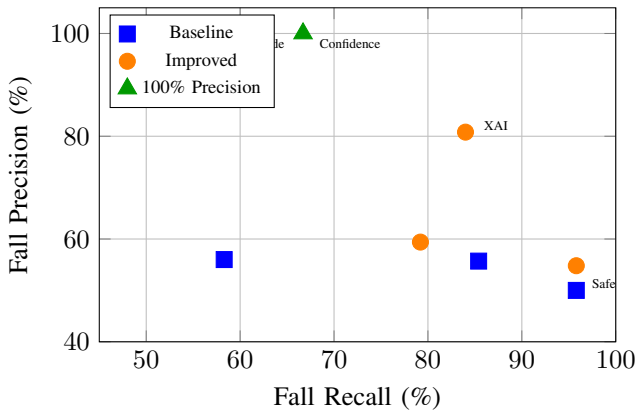

Fig. 6: Recall-Precision Trade-off. Green triangles indicate strategies achieving 100% precision (zero false positives).

E. Per-Dataset Analysis

Performance varies significantly across datasets, revealing dataset-specific challenges:

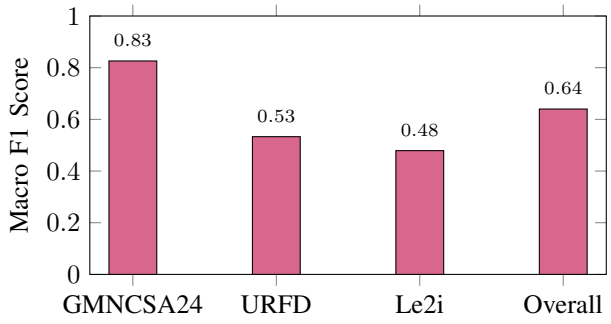

Fig. 7: Per-Dataset Macro F1 Scores. GMNCSA24 achieves highest performance (0.826), while Le2i presents the greatest challenge (0.479).

Analysis:

- **GMNCSA24** (0.826): Modern recording quality and clear fall events lead to best performance
- **URFD** (0.533): Fixed overhead camera provides clear view but limited data
- **Le2i** (0.479): Complex home environment with activities resembling falls (sitting, lying voluntarily) causes confusion

F. Model Comparison: GPT-4o vs GPT-4o-mini

TABLE IX: Model Performance Comparison

Model	Strategy	Acc	Macro F1	Cost/1K
GPT-4o-mini	Zero-Shot	66.7%	0.561	$0.15
GPT-4o-mini	Few-Shot	66.7%	0.656	$0.15
GPT-4o	Enhanced	75.4%	0.707	$2.50
GPT-4o	RAG	67.8%	0.678	$2.50
GPT-4o	Safety-First	57.9%	0.556	$2.50
DeepSeek	Few-Shot	64.9%	0.644	$0.07

Finding: GPT-4o improves accuracy by 8.7% over GPT-4o-mini (75.4% vs 66.7%) but costs 17x more. For deployment, the choice depends on the acceptable cost-performance trade-off.

VI. Discussion

A. Key Insights

1. Aggregation Rules Matter More Than Prompting Strategy

Our most surprising finding is that optimizing the video-level aggregation rule yields larger improvements than sophisticated prompting strategies. The same window-level predictions can produce Macro F1 scores ranging from 0.556 to 0.846 (a 52% improvement) simply by changing the aggregation rule from a simple ratio threshold to confidence-weighted aggregation.

Implication: Researchers should focus significant effort on aggregation strategy design, not just improving window-level classifiers.

2. Safety-First Design Achieves Near-Perfect Recall

Our safety-first prompting strategy achieves 95.8% recall, meaning only 1 in 24 falls is missed. This is achieved by embedding asymmetric cost considerations directly into the prompt:

> *"Missing a fall could be life-threatening. When in doubt, classify as FALL. It is better to have a false alarm than to miss a real fall."*

3. Confidence Scores Enable Precision Optimization

Token-level log probabilities from the LLM provide valuable uncertainty estimates. By summing confidence scores across windows and tuning a threshold, we achieve 100% precision while maintaining 66.7% recall—eliminating all false positives.

4. Two-Stage Cascades Balance Both Objectives

The cascade approach demonstrates that high recall and high precision can be achieved by separating the objectives across stages:

- Stage 1: Maximize recall (flag all potential falls)
- Stage 2: Maximize precision (filter false positives)

5. Dataset Quality Significantly Impacts Results

Le2i presents the greatest challenge (Macro F1 = 0.479), highlighting that pose-based features alone may be insufficient for distinguishing controlled descents (sitting, lying voluntarily) from actual falls. Additional context such as audio or environmental information may be needed.

B. Limitations

1) **Limited Test Set Size:** With only 57 test videos (24 falls, 33 no-falls), statistical power is limited and confidence intervals are wide. Larger datasets would strengthen conclusions.
2) **Pose Detection Failures:** MediaPipe occasionally fails to detect poses, particularly when subjects are partially occluded or in unusual positions. Failed frames were excluded, potentially biasing results.

3) **Single-Person Assumption:** The current system assumes single-person videos. Multi-person scenarios would require additional tracking logic.
4) **Real-Time Performance:** Processing a 10-second clip takes 30-90 seconds with current API calls—insufficient for real-time deployment. Edge deployment with local models would be needed.
5) **Cost:** GPT-4o API calls are expensive for large-scale deployment. Fine-tuning smaller models or using open-source LLMs could reduce costs.

C. Deployment Recommendations

Based on our findings, we recommend the following deployment configurations:

High-Safety Environment (Hospitals, Care Facilities):

- Strategy: Safety-First with ratio threshold 0.25
- Expected: 95.8% recall, 50% precision
- Use case: When missing a fall is unacceptable

Balanced Environment (Home Monitoring):

- Strategy: Confidence-Weighted Aggregation
- Expected: 66.7% recall, 100% precision
- Use case: When false alarms cause user distrust

Resource-Constrained Environment:

- Strategy: DeepSeek Few-Shot
- Expected: 62.5% recall, reasonable accuracy
- Use case: When API costs must be minimized

VII. Conclusion

This paper presented the first comprehensive evaluation of Generative AI techniques for video-based fall detection. Our key contributions and findings include:

1) **Novel Pipeline:** We developed a multimodal pipeline that converts video pose sequences into structured text for LLM classification, demonstrating that LLMs can effectively analyze biomechanical data for safety-critical applications.
2) **Strategy Comparison:** We evaluated 11 prompting strategies, finding that safety-first prompting achieves 95.8% recall while confidence-weighted aggregation achieves 100% precision with Macro F1 of 0.846.
3) **Aggregation Importance:** We demonstrated that video-level aggregation rules have more impact than prompting strategies, with optimized rules improving Macro F1 by 52%.
4) **Practical Insights:** We provided deployment recommendations for different use cases, balancing safety requirements with practical constraints.

A. Future Work

- **Larger Window Sizes:** Evaluate 5, 7, 9-frame windows for extended temporal context
- **Multi-Modal Fusion:** Combine pose features with audio cues (impact sounds) or depth information
- **Open-Source LLMs:** Evaluate Llama-3, Mistral, and other open models for cost reduction
- **Real-Time Deployment:** Optimize pipeline for edge devices using quantized models
- **Active Learning:** Use LLM uncertainty estimates to prioritize annotation of ambiguous cases
- **Multi-Person Scenarios:** Extend pipeline to handle multiple subjects with tracking

Acknowledgments

We express our sincere gratitude to **Professor Dr. Mohammad Masum** for his invaluable guidance, insightful feedback, and constant encouragement throughout this project. His expertise in Generative AI and dedication to student success have been instrumental in shaping this research.

We are also deeply grateful to our Teaching Assistant, **Mayank Kapadia**, for his continuous support, prompt assistance with technical challenges, and constructive suggestions that significantly improved our work.

This project was completed as part of the DATA 266: Generative AI course at San Jose State University, Spring 2026.

References

[1] World Health Organization, "Falls," 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/falls

[2] Centers for Disease Control and Prevention, "Important Facts about Falls," 2020. [Online]. Available: https://www.cdc.gov/falls/facts.html

[3] N. Noury, A. Fleury, P. Rumeau, A. K. Bourke, G. O. Laighin, V. Rialle, and J. E. Lundy, "Fall detection-principles and methods," in *Proc. 29th IEEE Engineering in Medicine and Biology Society (EMBC)*, 2007, pp. 1663–1666.

[4] M. Mubashir, L. Shao, and L. Seed, "A survey on fall detection: Principles and approaches," *Neurocomputing*, vol. 100, pp. 144–152, 2013.

[5] C. Rougier, J. Meunier, A. St-Arnaud, and J. Rousseau, "Robust video surveillance for fall detection based on human shape deformation," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 5, pp. 611–622, 2011.

[6] S. S. Khan and J. Hoey, "Review of fall detection techniques: A data availability perspective," *Medical Engineering & Physics*, vol. 39, pp. 12–22, 2017.

[7] D. Chen, Y. Feng, H. Zhang, and Y. Wei, "Fall detection based on pose estimation and motion history image," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2020.

[8] K. Chen and G. Tan, "Pose-based fall detection using LSTM networks," in *Proc. ACM International Conference on Multimedia (MM)*, 2020.

[9] B. Kwolek and M. Kepski, "Human fall detection on embedded platform using depth maps and wireless accelerometer," *Computer Methods and Programs in Biomedicine*, vol. 117, no. 3, pp. 489–501, 2014.

[10] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[11] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson, "Large language models are zero-shot time series forecasters," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[12] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[13] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[15] V. R. Seelam, H. Boinepally, and Shristi, "GenAI Fall Detection Project," 2026. [Online]. Available: https://github.com/Ramreddy2748/Gen_AI_Project